



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2025-2026

Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

IndicSearch : An Efficient Keyword-Based Search Engine

Mydam Aswitha Patel

2520030001

Gutha Sushma Sree

2520030098

N. Yishnaani

2520030414

COURSE INSTRUCTOR

Dr. J Sirisha Devi

Professor

Department of Computer Science and Engineering

IndicSearch: An Efficient Keyword-Based Search Engine for Indian-Language Wikipedia

Brief Description of the Case Study

IndicSearch is a DSA-based keyword search engine designed to retrieve relevant information from Indian-language Wikipedia content. The project addresses the challenge of searching multilingual text efficiently while correctly handling Unicode characters. Wikipedia is used as the real data source, while Java-based data structures and algorithms perform text processing, keyword retrieval, and result ranking.

Searching a large collection of documents by scanning every document for every query can be inefficient. This becomes more challenging when the content is multilingual because Indian languages such as Hindi, Telugu, Tamil, Kannada, Malayalam, Bengali, Marathi, Gujarati, and Punjabi use different scripts and require proper Unicode/UTF-8 handling. Users also expect search results to be relevant and useful rather than being displayed in an arbitrary order.

Problem and Proposed Solution

Consider a user searching for the query “Indian Cricket”. A simple search approach could check every available document for the presence of the words “Indian” and “Cricket”. As the number of documents increases, repeatedly scanning all documents becomes less efficient. IndicSearch addresses this problem by applying an Inverted Index that maps keywords to the documents in which they occur.

For example, an index may contain: cricket → Document 1, Document 2, Document 4; Indian → Document 1, Document 3. When the user searches for multiple keywords, the system retrieves matching candidate documents and combines keyword frequencies to calculate document scores. The results are then ranked using Merge Sort.

Major DSA Concepts Used

HashMap: HashMap is used for efficient key-value based storage and retrieval. In the project, it manages documents and keyword-frequency or score information. This provides efficient access to the data required during searching.

Inverted Index: The Inverted Index maintains a mapping from a keyword to the documents containing that keyword. For example, “cricket” can point to several

matching documents. This allows the search engine to retrieve candidate documents directly instead of scanning all available documents for every query.

Trie: A Trie is used for prefix-based search suggestions. When a user types a prefix such as “cri”, words beginning with that prefix can be identified and suggested. This improves the interactive search experience.

Merge Sort: After matching documents are obtained, Merge Sort is used to arrange the results according to their calculated keyword frequency or score. Documents with higher frequency can therefore be displayed earlier in the ranked results.

Text Preprocessing and Unicode Support

Before searching, the query and retrieved text are normalized and tokenized. Normalization converts text into a consistent form, while tokenization divides a sentence into individual words. For example, “Indian Cricket Team” can be processed into the tokens “Indian”, “Cricket”, and “Team”. The system also supports Unicode/UTF-8 text, allowing queries to be entered in Indian languages such as “भारत” or “ತೆಲಂಗಾಣ”.

Example of the Complete Search Process

Suppose a user enters the query “Indian Cricket” and selects a Wikipedia language. The system first preprocesses the query and obtains the keywords “Indian” and “Cricket”. The Wikipedia MediaWiki API then retrieves candidate pages. The Java search engine processes these documents using the HashMap and Inverted Index, identifies matching keywords, and calculates a frequency-based score.

For example, if “Indian cricket team” has a score of 8, “Indian Premier League” has a score of 5, “Cricket World Cup” has a score of 3, and “History of cricket” has a score of 2, Merge Sort arranges them in descending score order. The ranked results are then displayed through the web interface.

Prefix Search Example

The project also provides prefix suggestions using Trie. If the user types “hyder”, the system can provide matching suggestions such as “Hyderabad” or other relevant prefix matches. The basic process is: User enters a prefix → Trie performs prefix matching → Search suggestions are displayed.

Overall System Flow

The overall flow is: User → Enter Query → Select Language → Text Preprocessing → Wikipedia MediaWiki API → Retrieve Candidate Documents → HashMap and Inverted Index → Keyword Matching → Calculate Frequency Scores → Merge Sort → Rank Results → Display Results. For autocomplete, the separate flow is User Types Prefix → Trie → Prefix Suggestions.

Real-World Relevance and Expected Outcome

IndicSearch demonstrates how fundamental Data Structures and Algorithms can be applied to a practical information-retrieval problem. The project combines real Wikipedia data with Java-based DSA processing rather than relying only on predefined sample records. It provides a working web interface for keyword searching, result ranking, prefix suggestions, and search-performance information.

The expected outcome is a working multilingual Wikipedia search system that performs keyword-based retrieval, frequency-based ranking, and prefix suggestions using DSA techniques while supporting Unicode text.

Future Scope

The system can be extended to support a larger locally indexed multilingual corpus, improve ranking using additional relevance factors, add more Indian languages, enhance autocomplete and query understanding, and introduce caching for faster repeated searches.

ALGORITHM / PSEUDO CODE

1. Query Handling – ApiServer

The ApiServer receives the search request through the /api/search endpoint. It reads the query parameter q and the language parameter lang. If the language is not supplied, the project uses English as the default.

PSEUDOCODE

FUNCTION handleSearch(request)

 Read query from parameter "q"

 Read language from parameter "lang"

 IF language is empty

 language = "en"

 END IF

 IF query is empty

 return empty response

 END IF

 Continue to Wikipedia search

END FUNCTION

2. Retrieve Candidate Pages – WikipediaClient

The WikipediaClient constructs a MediaWiki API search URL using the selected language and user query. It sends the request using Java HttpClient and receives candidate Wikipedia search results. Each result contains an ID, title, snippet, URL, and word count.

PSEUDOCODE

FUNCTION search(query, language)

 IF query is empty

 return empty list

 END IF

```
IF language is empty
    language = "en"
END IF

Encode query using UTF-8

Create Wikipedia API URL

Send HTTP GET request using HttpClient

IF response status is not successful
    handle API error / rate limit
END IF

Parse title, snippet and word count

Create WikipediaResult objects

Return candidate results

END FUNCTION
```

3. Text Preprocessing – TextPreprocessor

The TextPreprocessor class normalizes text using Unicode NFC normalization, converts text to lowercase, replaces punctuation and symbols with spaces, removes extra whitespace, and tokenizes the resulting text.

PSEUDOCODE

```
FUNCTION normalize(text)

    IF text is null
        return ""
    END IF

    Apply Unicode NFC normalization

    Convert text to lowercase

    Replace punctuation and symbols with spaces

    Replace multiple spaces with one space

    Trim leading and trailing spaces
```

```

    Return normalized text
END FUNCTION

FUNCTION tokenize(text)
    normalized = normalize(text)

    IF normalized is empty
        return empty list
    END IF

    Split normalized text using whitespace

    Return tokens
END FUNCTION

```

4. Store Documents and Build DSA Structures – SearchEngine

For every WikipediaResult returned by WikipediaClient, ApiServer creates a Document and adds it to a new SearchEngine instance. The SearchEngine stores the document in a HashMap. The implemented SearchEngine also combines the document title and retrieved text before passing it to the Inverted Index and Trie.

PSEUDOCODE

```

FUNCTION addDocument(document)

    Store document in documents HashMap using document ID

    searchableText =

        document.title + " " + document.text

    Create searchable document using searchableText

    Add searchable document to InvertedIndex

    tokens = TextPreprocessor.tokenize(searchableText)

    FOR each word in tokens

        Insert word into Trie
    END FOR

```

END FUNCTION

5. Build the Inverted Index – InvertedIndex

The InvertedIndex maps each normalized word to a HashMap containing document IDs and their occurrence frequencies. Every time a word occurs in a document, its frequency for that document is increased.

PSEUDOCODE

FUNCTION addDocument(document)

 words = TextPreprocessor.tokenize(document.text)

 FOR each word in words

 IF index does not contain word

 Create new HashMap for word

 END IF

 documents = index[word]

 currentFrequency =

 documents.getDefault(document.id, 0)

 documents[document.id] =

 currentFrequency + 1

 END FOR

END FUNCTION

FUNCTION searchWord(word)

 word = TextPreprocessor.normalize(word)

 Return index[word] or an empty HashMap

END FUNCTION

6. Search and Calculate Document Scores – SearchEngine

When search() is called, the query is normalized and split into keywords. For each keyword, the Inverted Index returns matching document IDs and

frequencies. The SearchEngine adds the frequencies of all query keywords to create a score for each document.

PSEUDOCODE

FUNCTION search(query)

 start timer

 normalizedQuery =

 TextPreprocessor.normalize(query)

 IF normalizedQuery is empty

 return empty list

 END IF

 keywords = split normalizedQuery by whitespace

 Create empty scores HashMap

 FOR each keyword in keywords

 matches = invertedIndex.searchWord(keyword)

 FOR each (documentID, frequency) in matches

 scores[documentID] =

 scores.getDefault(documentID, 0)

 + frequency

 END FOR

 END FOR

 Create empty SearchResult list

 FOR each (documentID, score) in scores

 document = documents[documentID]

 IF document exists

 Add SearchResult(document, score)

 END IF

 END FOR

Sort results using MergeSort

Stop timer

Store search metrics

Return results

END FUNCTION

7. Rank Results – MergeSort

The project uses the implemented MergeSort class to sort SearchResult objects according to their frequency. During merging, the result with the higher frequency is placed first, producing descending frequency order.

PSEUDOCODE

FUNCTION sort(results)

IF size of results $\leq$ 1

return

END IF

mid = size / 2

left = first half

right = second half

sort(left)

sort(right)

merge(results, left, right)

END FUNCTION

FUNCTION merge(results, left, right)

Compare left.frequency and right.frequency

Place the higher frequency result first

Copy remaining elements from left or right

Continue until all results are merged

END FUNCTION

8. Prefix Data Structure – Trie

The project's SearchEngine inserts every processed word into the Trie. The Trie stores characters through child nodes and marks complete words. Its getSuggestions() method follows the entered prefix and recursively collects stored words beginning with that prefix.

PSEUDOCODE

FUNCTION insert(word)

 current = root

 FOR each character ch in word

 IF ch is not a child of current

 Create a new TrieNode

 END IF

 current = current.children[ch]

 END FOR

 current.isEnd = true

END FUNCTION

FUNCTION getSuggestions(prefix)

 current = root

 FOR each character ch in prefix

 IF ch is not present

 return empty list

 END IF

 current = current.children[ch]

 END FOR

 Collect words below current node

 Return up to 5 suggestions

END FUNCTION

9. Suggestions Endpoint – ApiServer / WikipediaClient

The current ApiServer calls WikipediaClient.suggestions() for the live /api/suggestions endpoint. WikipediaClient uses the MediaWiki prefixsearch API to obtain suggestion titles. The Java Trie is built inside SearchEngine for prefix searching, but the current live suggestions endpoint retrieves its displayed suggestions directly from Wikipedia's prefixsearch API. This distinction is kept here so the documentation matches the actual code.

PSEUDOCODE

FUNCTION handleSuggestions(request)

 Read prefix

 Read language

 Call wikipediaClient.suggestions(prefix, language)

 Return suggestion titles as JSON

END FUNCTION

10. Combine Results and Return JSON – ApiServer

After [SearchEngine.search\(\)](#) returns ranked SearchResult objects, ApiServer combines the DSA-ranked results with the Wikipedia candidate results and creates the JSON response. The response contains query information, language, search metrics, algorithm information, and result details.

PSEUDOCODE

FUNCTION handleSearch(request)

 wikiResults = wikipediaClient.search(query, language)

 Create new SearchEngine

 FOR each WikipediaResult

 Create Document

 engine.addDocument(document)

 END FOR

 rankedResults = engine.search(query)

Get search metrics from engine

Prepare JSON response

Return response to frontend

END FUNCTION

11. Complete Project Algorithm

ALGORITHM INDICSEARCH(query, language)

1. Receive query and selected language from the frontend.
2. Validate the query.
3. Normalize and tokenize text using TextPreprocessor.
4. Send the query to Wikipedia through WikipediaClient.
5. Receive candidate Wikipedia pages.
6. Create Document objects for the retrieved pages.
7. Create a SearchEngine instance.
8. Store documents in the SearchEngine HashMap.
9. Combine title and retrieved text for indexing.
10. Tokenize the searchable text.
11. Add words and document frequencies to the Inverted Index.
12. Insert processed words into the Java Trie.
13. Normalize the user's query.
14. Split the query into individual keywords.
15. Search each keyword in the Inverted Index.
16. Add keyword frequencies to document scores.
17. Create SearchResult objects for matched documents.
18. Apply Merge Sort to rank results by frequency.
19. Collect search-performance metrics.

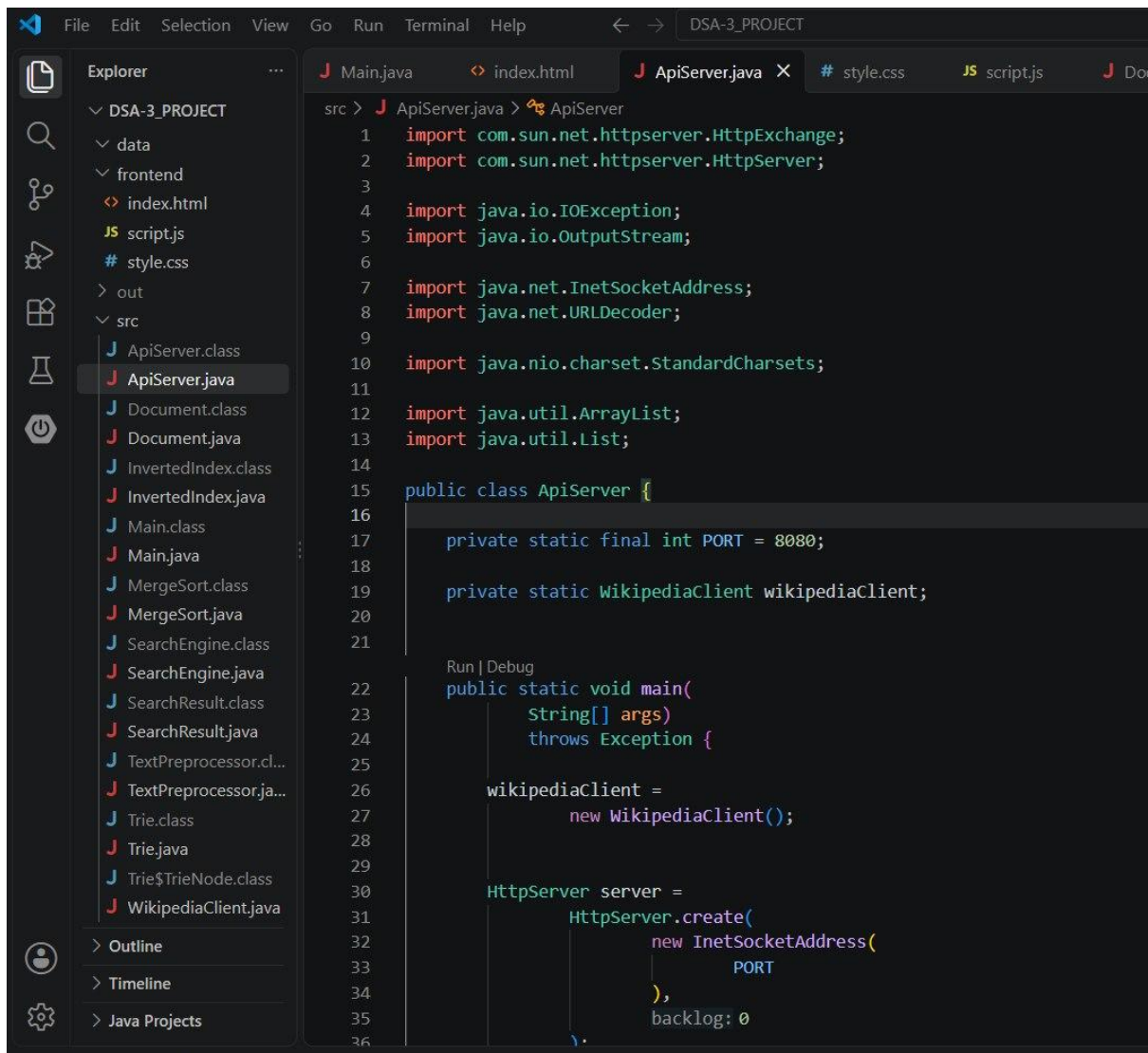
20. Return the ranked results and metrics through ApiServer.
21. For the current live suggestion endpoint, obtain prefix suggestions through Wikipedia's prefixsearch API.
22. Display the results in the web frontend.

END ALGORITHM

12. Final Flow

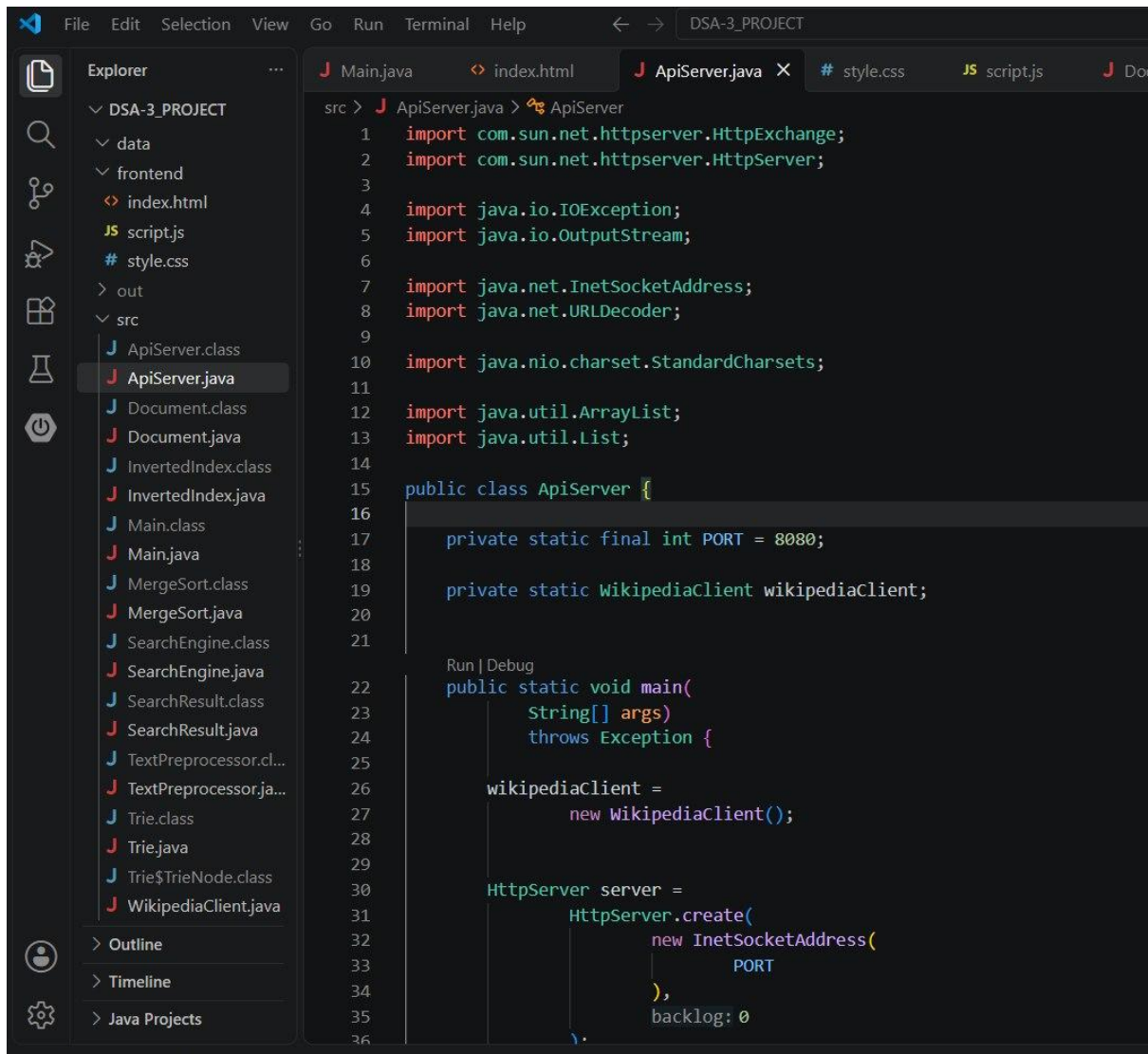
Frontend → ApiServer → WikipediaClient → Candidate Wikipedia Results → Document Creation → SearchEngine → HashMap + Inverted Index + Trie → Keyword Matching → Frequency Scores → Merge Sort → Ranked Results → JSON Response → Frontend

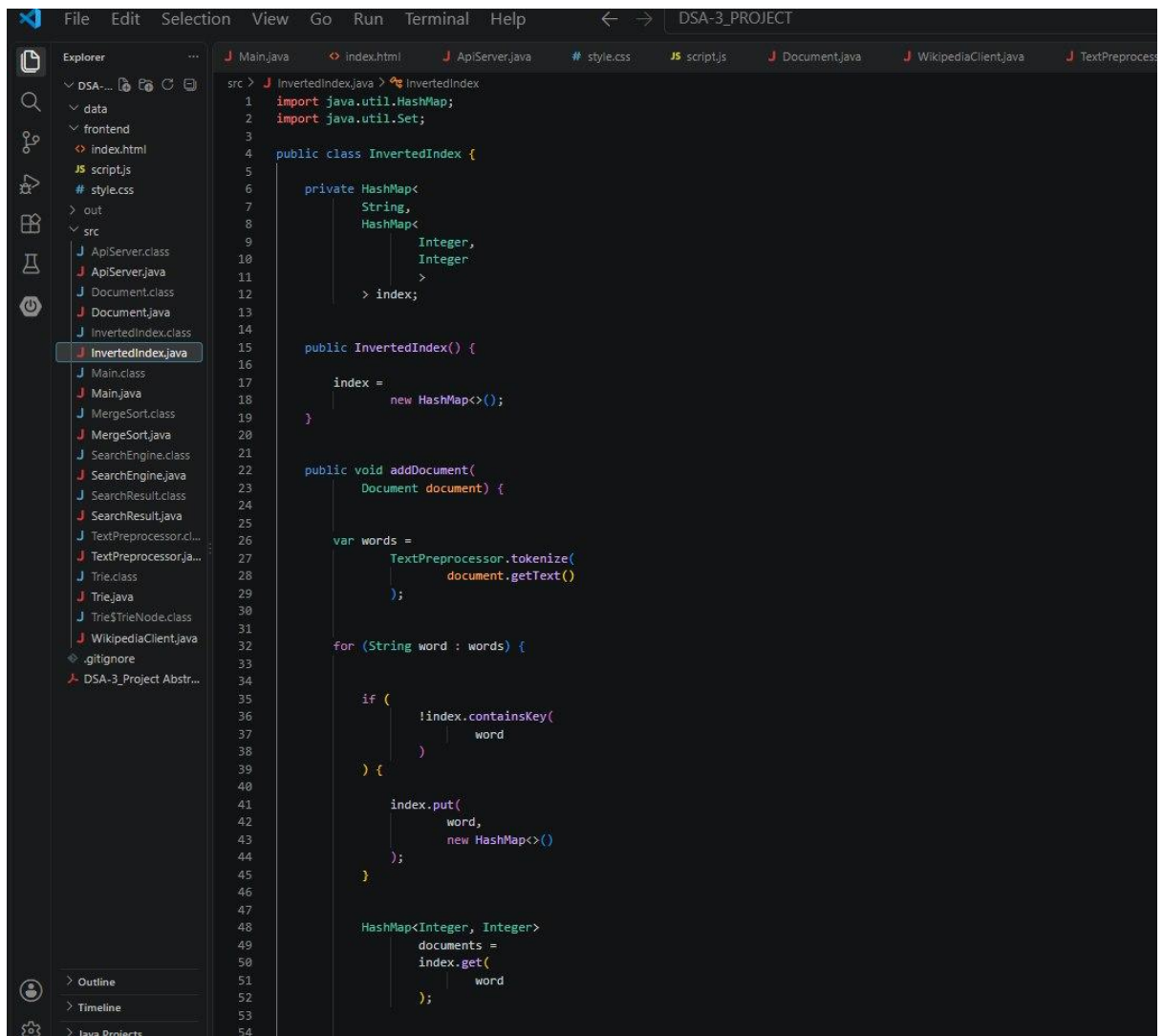
CODE



The image shows a screenshot of an IDE (IntelliJ IDEA) with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The title bar shows 'DSA-3_PROJECT'. The Explorer panel on the left displays a project structure for 'DSA-3_PROJECT' with folders 'data', 'frontend', 'out', and 'src'. The 'src' folder is expanded, showing files like 'ApiServer.class', 'ApiServer.java', 'Document.class', 'Document.java', 'InvertedIndex.class', 'InvertedIndex.java', 'Main.class', 'Main.java', 'MergeSort.class', 'MergeSort.java', 'SearchEngine.class', 'SearchEngine.java', 'SearchResult.class', 'SearchResult.java', 'TextPreprocessor.cl...', 'TextPreprocessor.ja...', 'Trie.class', 'Trie.java', 'Trie\$TrieNode.class', and 'WikipediaClient.java'. The 'ApiServer.java' file is selected. The main editor area shows the code for 'ApiServer.java'. The code includes imports for 'com.sun.net.httpserver.HttpExchange', 'com.sun.net.httpserver.HttpServer', 'java.io.IOException', 'java.io.OutputStream', 'java.net.InetSocketAddress', 'java.net.URLDecoder', 'java.nio.charset.StandardCharsets', 'java.util.ArrayList', and 'java.util.List'. The code defines a 'public class ApiServer' with a 'private static final int PORT = 8080;', a 'private static WikipediaClient wikipediaClient;', and a 'public static void main(String[] args) throws Exception' method. The 'main' method initializes 'wikipediaClient' as a new 'WikipediaClient()' and creates an 'HttpServer' using 'HttpServer.create(new InetSocketAddress(PORT), backlog: 0)'. The code is displayed with line numbers from 1 to 36.

```
src > J ApiServer.java > ApiServer
1  import com.sun.net.httpserver.HttpExchange;
2  import com.sun.net.httpserver.HttpServer;
3
4  import java.io.IOException;
5  import java.io.OutputStream;
6
7  import java.net.InetSocketAddress;
8  import java.net.URLDecoder;
9
10 import java.nio.charset.StandardCharsets;
11
12 import java.util.ArrayList;
13 import java.util.List;
14
15 public class ApiServer {
16
17     private static final int PORT = 8080;
18
19     private static WikipediaClient wikipediaClient;
20
21
22     public static void main(
23         String[] args)
24         throws Exception {
25
26         wikipediaClient =
27             new WikipediaClient();
28
29
30         HttpServer server =
31             HttpServer.create(
32                 new InetSocketAddress(
33                     PORT
34                 ),
35                 backlog: 0
36             );
37     }
38 }
```



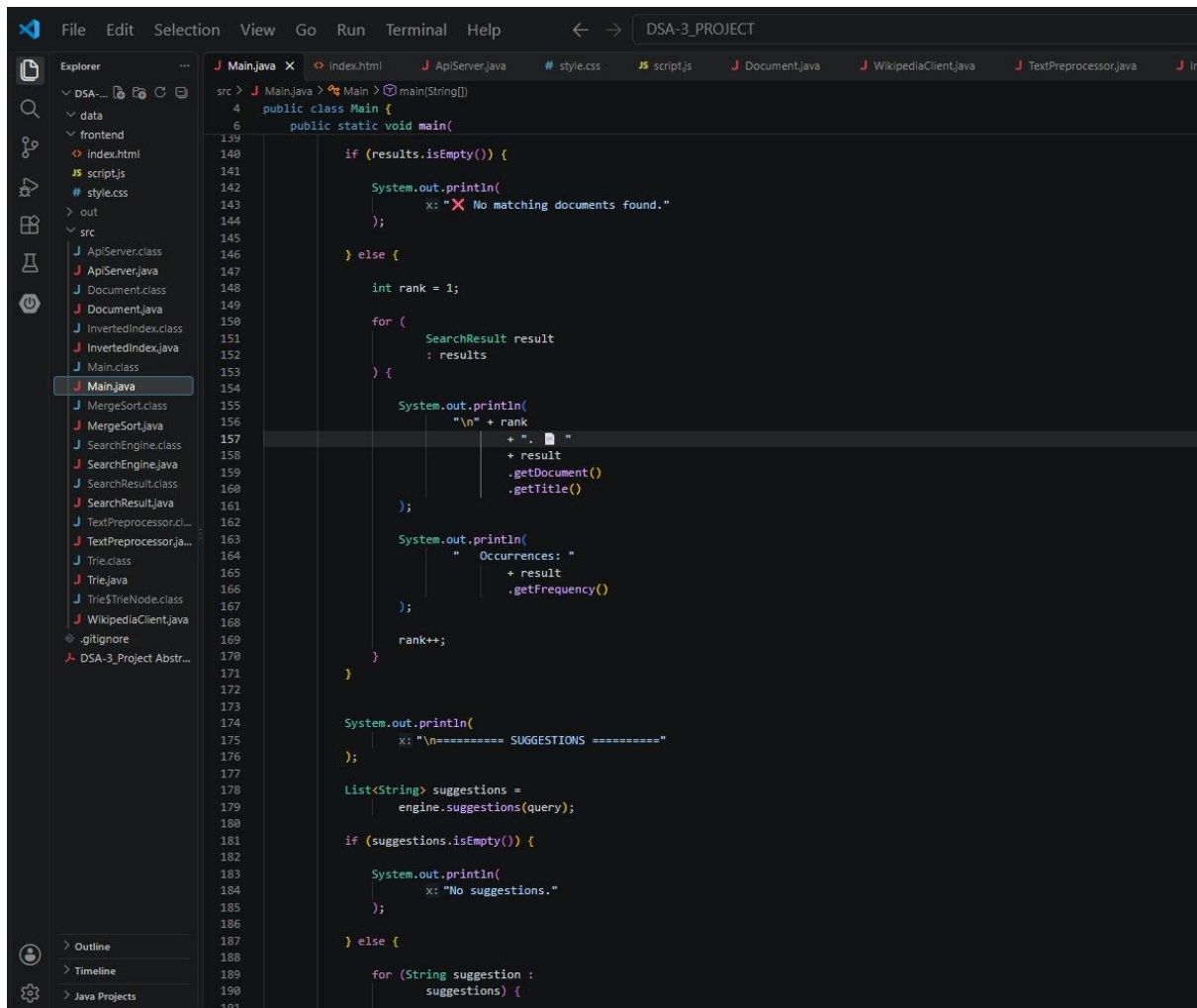


```
File Edit Selection View Go Run Terminal Help
DSA-3_PROJECT

src > J Main.java > Main > main(String[])
4 public class Main {
6     public static void main(
8         String[] args) {
10
11         SearchEngine engine =
12             new SearchEngine();
13
14         // =====
15         // SAMPLE WIKIPEDIA ARTICLES
16         // =====
17
18         engine.addDocument(
19             new Document(
20                 id: 1,
21                 title: "भारत",
22                 "भारत दक्षिण एशिया में स्थित "
23                 + "एक देश है। भारत की राजधानी "
24                 + "नई दिल्ली है। भारत एक विविध "
25                 + "संस्कृति वाला देश है।"
26             )
27         );
28
29         engine.addDocument(
30             new Document(
31                 id: 2,
32                 title: "भारतीय क्रिकेट",
33                 "क्रिकेट भारत में बहुत लोकप्रिय "
34                 + "खेल है। भारतीय क्रिकेट टीम ने "
35                 + "कई अंतरराष्ट्रीय मैच जीते हैं। "
36                 + "भारत में क्रिकेट के बहुत "
37                 + "प्रशंसक हैं।"
38             )
39         );
40
41         engine.addDocument(
42             new Document(
43                 id: 3,
44                 title: "हैदराबाद",
45                 "हैदराबाद भारत के तेलंगाना "
46                 + "राज्य का एक प्रमुख शहर है। "
47                 + "हैदराबाद अपनी संस्कृति और "
48                 + "तकनीकी उद्योग के लिए "
49                 + "प्रसिद्ध है।"
50             )
51         );
52
53     }
```

```
File Edit Selection View Go Run Terminal Help
DSA-3_PROJECT

src > J Main.java > Main > main(String[])
76     };
77
78     Scanner scanner =
79         new Scanner(System.in);
80
81     System.out.println(
82         x: "=====
83     );
84
85     System.out.println(
86         x: " IN BHARAT SMART SEARCH"
87     );
88
89     System.out.println(
90         x: " Indian Wikipedia Text Search Engine"
91     );
92
93     System.out.println(
94         x: "=====
95     );
96
97     while (true) {
98
99         System.out.print(
100             "\n Enter keyword "
101             + "(type exit to stop): "
102         );
103
104         String query =
105             scanner.nextLine();
106
107         if (
108             query.equalsIgnoreCase(
109                 anotherString: "exit"
110             )
111         ) {
112             break;
113         }
114
115         if (query.trim().isEmpty()) {
116
117             System.out.println(
118                 x: "Please enter a keyword."
119             );
120
121         }
122
123     }
```



The image shows a screenshot of an IDE (likely IntelliJ IDEA) with a dark theme. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. The title bar shows "DSA-3_PROJECT". The Explorer panel on the left displays a project structure with folders like "data", "frontend", "out", and "src". The "src" folder is expanded, showing various Java files including "Main.java", which is currently selected. The main editor area displays the code for "Main.java". The code defines a "Main" class with a "main" method. It checks if "results" is empty and prints a message if so. Otherwise, it iterates over "results", printing the rank, document title, and frequency for each. It also prints a separator for suggestions and calls "engine.suggestions(query)". If there are no suggestions, it prints "No suggestions."; otherwise, it iterates over the suggestions.

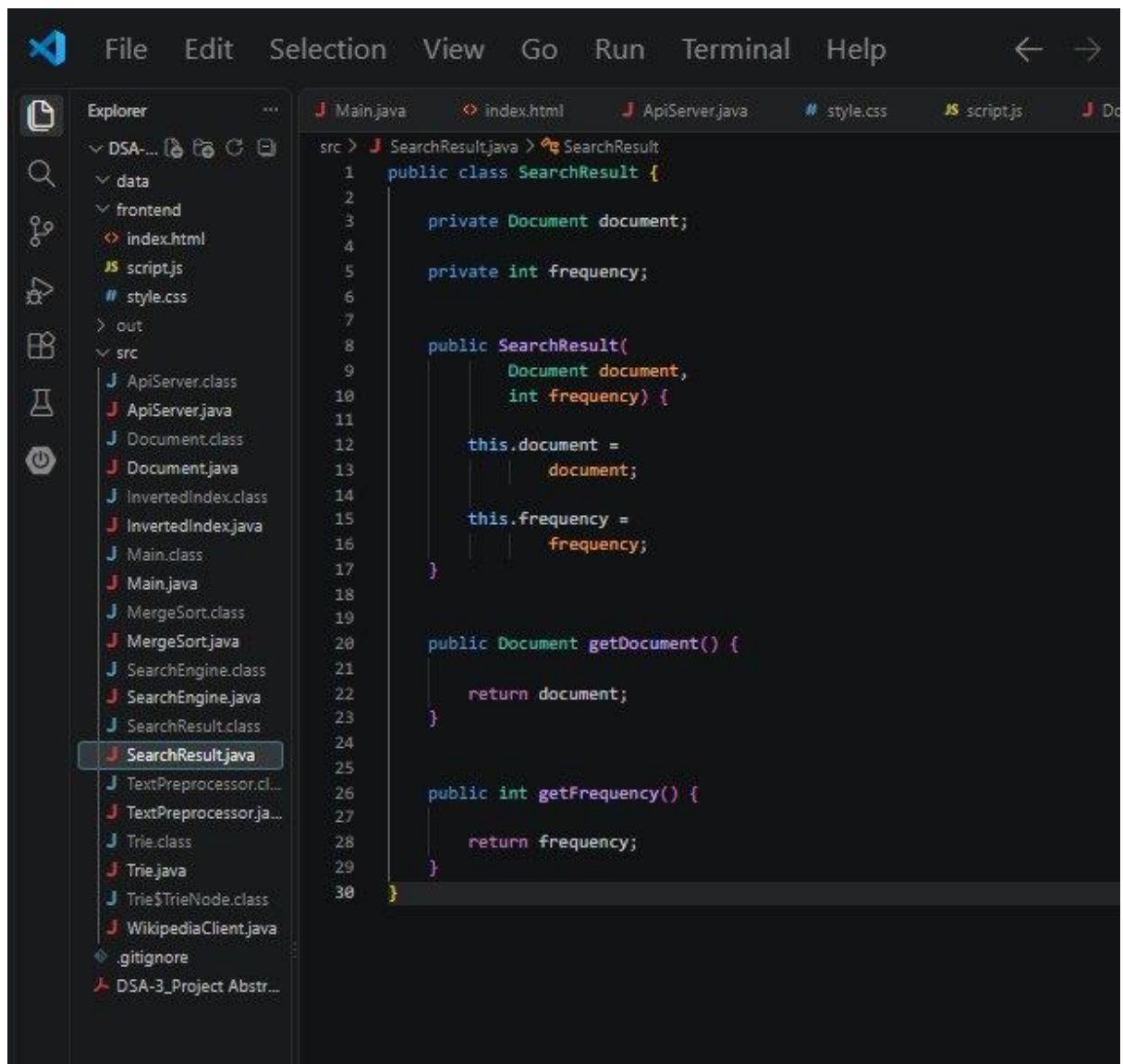
```
src > J Main.java X Main > main(String[])
4 public class Main {
6     public static void main(
139
140         if (results.isEmpty()) {
141
142             System.out.println(
143                 x: "X No matching documents found."
144             );
145         } else {
146
147             int rank = 1;
148
149             for (
150                 SearchResult result
151                 : results
152             ) {
153
154                 System.out.println(
155                     "\n" + rank
156                     + ".  "
157                     + result
158                     .getDocument()
159                     .getTitle()
160                 );
161
162                 System.out.println(
163                     "    Occurrences: "
164                     + result
165                     .getFrequency()
166                 );
167
168                 rank++;
169             }
170
171         }
172
173         System.out.println(
174             x: "\n===== SUGGESTIONS ====="
175         );
176
177         List<String> suggestions =
178             engine.suggestions(query);
179
180         if (suggestions.isEmpty()) {
181
182             System.out.println(
183                 x: "No suggestions."
184             );
185
186         } else {
187
188             for (String suggestion :
189                 suggestions) {
190
191
```

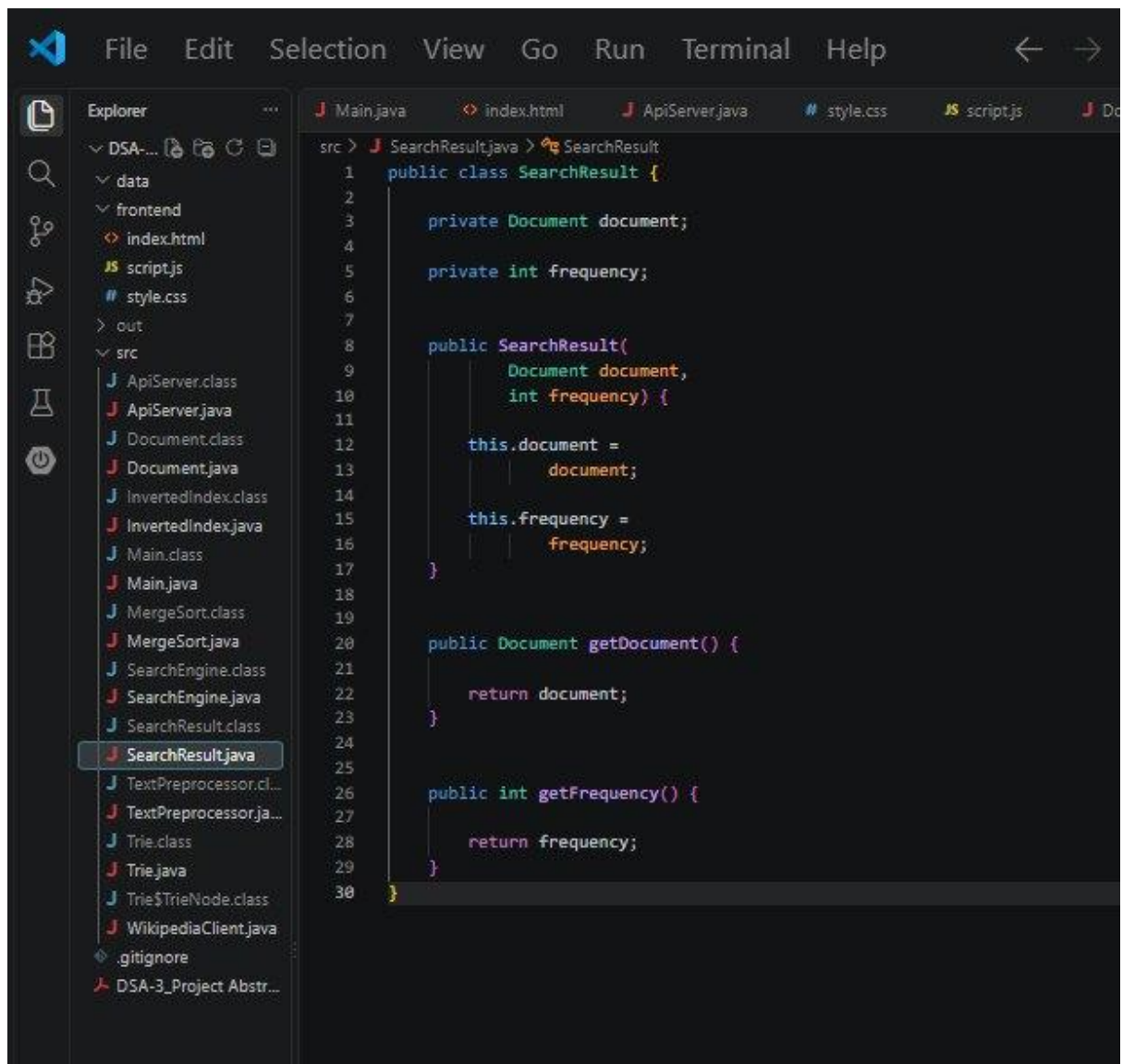
The screenshot shows an IDE window titled "DSA-3_PROJECT". The Explorer sidebar on the left displays a project structure with folders "data", "frontend", "out", and "src". The "src" folder contains several Java files, with "Main.java" selected. The main editor displays the code for "Main.java", which includes a "main" method. The code logic is as follows: it checks if "results" is empty; if so, it prints "No matching documents found."; otherwise, it iterates through "results" (ranked from 1), printing each result's document and title, followed by the number of occurrences. Finally, it prints a section for suggestions, which are retrieved from an "engine" and printed if they exist.

```
src > J Main.java X Main > main(String[])
4 public class Main {
6     public static void main(
139
140         if (results.isEmpty()) {
141
142             System.out.println(
143                 x: "X No matching documents found."
144             );
145         } else {
146
147             int rank = 1;
148
149             for (
150                 SearchResult result
151                 : results
152             ) {
153
154                 System.out.println(
155                     "\n" + rank
156                     + ".  "
157                     + result
158                     .getDocument()
159                     .getTitle()
160                 );
161
162                 System.out.println(
163                     "    Occurrences: "
164                     + result
165                     .getFrequency()
166                 );
167
168                 rank++;
169             }
170
171         }
172
173         System.out.println(
174             x: "\n===== SUGGESTIONS ====="
175         );
176
177         List<String> suggestions =
178             engine.suggestions(query);
179
180         if (suggestions.isEmpty()) {
181
182             System.out.println(
183                 x: "No suggestions."
184             );
185
186         } else {
187
188             for (String suggestion :
189                 suggestions) {
190
191
```

```
File Edit Selection View Go Run Terminal Help
DSA-3_PROJECT

src > SearchEngine.java > SearchEngine
1  import java.util.ArrayList;
2  import java.util.HashMap;
3  import java.util.List;
4  import java.util.Map;
5
6  public class SearchEngine {
7
8      private HashMap<Integer, Document> documents;
9
10     private InvertedIndex invertedIndex;
11
12     private Trie trie;
13
14     private double lastSearchTime;
15     private int lastMatchedDocuments;
16     private int lastKeywordCount;
17     private String lastAlgorithm;
18
19     public SearchEngine() {
20
21         documents = new HashMap<>();
22
23         invertedIndex = new InvertedIndex();
24
25         trie = new Trie();
26
27         lastSearchTime = 0.0;
28         lastMatchedDocuments = 0;
29         lastKeywordCount = 0;
30
31         lastAlgorithm =
32             "HashMap + Inverted Index + Merge Sort";
33     }
34
35     // =====
36     // ADD DOCUMENT
37     // =====
38
39     public void addDocument(
40         Document document) {
41
42         documents.put(
43             document.getId(),
44             document
45         );
46     }
47
48     /*
49     * IMPORTANT:
50     *
51     * Index BOTH the title and the text.
52     *
53     * Earlier we indexed only the snippet.
54     * That caused searches for names such as
55     * "Dr. Sambiah Mydam" to fail when the
56     * words were present in the title but
57     * not completely present in the snippet.
58     */
59
60     String searchableText =
61         document.getTitle()
62         + " "
63         + document.getText();
64
65 }
```

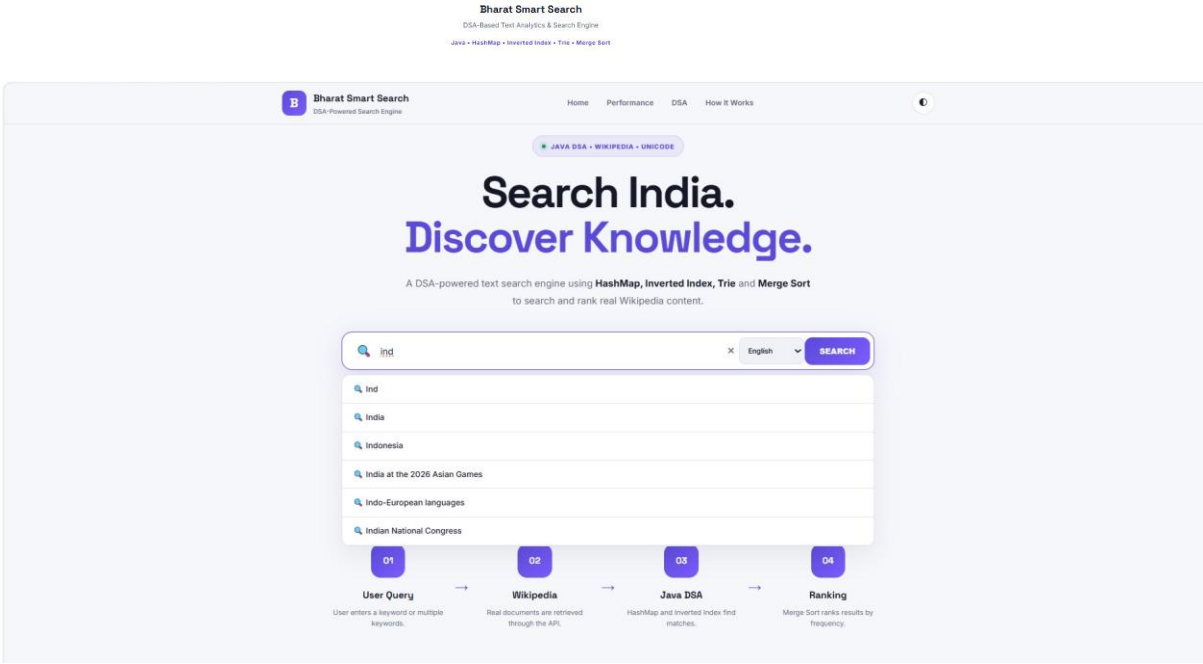
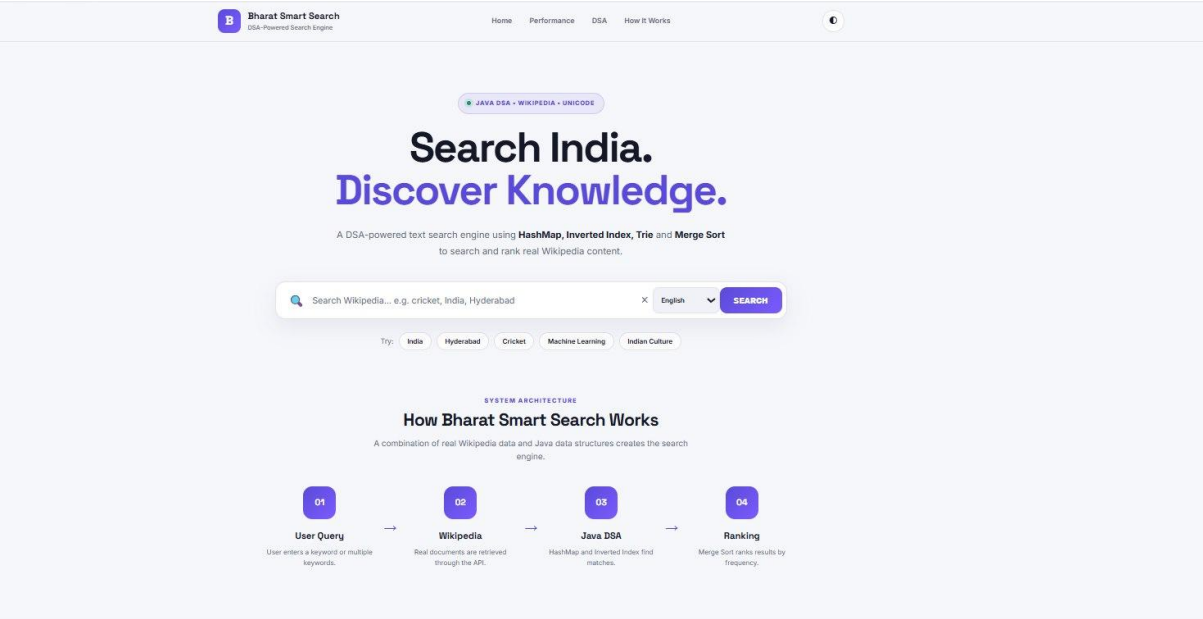




```
src > J TestPreprocessor.java > J TestPreprocessor
1  import java.text.Normalizer;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class TextPreprocessor {
7
8
9      public static String normalize(
10         String text) {
11
12         if (text == null) {
13             return "";
14         }
15
16         text =
17             Normalizer.normalize(
18                 text,
19                 Normalizer.Form.NFC
20             );
21
22         text =
23             text.toLowerCase();
24
25         text =
26             text.replaceAll(
27                 regex: "[\\p{P}\\p{S}]",
28                 replacement: ""
29             );
30
31         text =
32             text.replaceAll(
33                 regex: "\\s+",
34                 replacement: " "
35             );
36
37         return text.trim();
38     }
39
40     public static List<String> tokenize(
41         String text) {
42
43         String normalized =
44             normalize(text);
45
46         List<String> tokens =
47             new ArrayList<>();
48
49         if (
50             normalized.isEmpty()
51         ) {
52             return tokens;
53         }
54
55         return tokens;
56     }
57
58
59
60
61
62
63
64
65
66
```

```
src > J WikipediaClient.java > J WikipediaClient
1  import java.net.URI;
2  import java.net.URLEncoder;
3  import java.net.http.HttpClient;
4  import java.net.http.HttpRequest;
5  import java.net.http.HttpResponse;
6  import java.nio.charset.StandardCharsets;
7
8  import java.util.ArrayList;
9  import java.util.List;
10
11  public class WikipediaClient {
12
13      private final HttpClient client;
14
15      /*
16       * IMPORTANT:
17       * Use a meaningful User-Agent.
18       */
19      private static final String USER_AGENT =
20          "BharatSmartSearch/1.0 (KLM University student project)";
21
22      /*
23       * Number of times we retry after HTTP 429.
24       */
25      private static final int MAX_RETRIES = 3;
26
27      public WikipediaClient() {
28
29         client =
30             HttpClient.newBuilder()
31                 .followedRedirects(
32                     HttpClient.Redirect.NORMAL
33                 )
34                 .build();
35     }
36
37
38
39     // =====
40     // SEARCH WIKIPEDIA
41     // =====
42
43     public List<WikipediaResult> search(
44         String query,
45         String language)
46         throws Exception {
47
48         List<WikipediaResult> results =
49             new ArrayList<>();
50
51         if (query == null ||
52             query.trim().isEmpty()) {
53             return results;
54         }
55
56
57         if (language == null ||
58             language.trim().isEmpty()) {
59             language = "en";
60         }
61
62         /*
63          * First search using the exact query.
64          */
65         results =
66
```

RESULTS



B

Bharat Smart Search

DSA-Powered Search Engine

[Home](#)[Performance](#)[DSA](#)[How It Works](#)

JAVA DSA • WIKIPEDIA • UNICODE

Search India.

Discover Knowledge.

A DSA-powered text search engine using **HashMap**, **Inverted Index**, **Trie** and **Merge Sort** to search and rank real Wikipedia content.

India

X

English

SEARCH

Try: [India](#) [Hyderabad](#) [Cricket](#) [Machine Learning](#) [Indian Culture](#)

LIVE SEARCH ANALYTICS

Search Performance

Searching for "India"

1076.95ms

Total Search Time

Complete request time

0.170 ms

DSA Processing

Java algorithm execution

10

Documents Indexed

Wikipedia candidates

9

Matching Documents

Keyword matches

155

Indexed Keywords

Unique indexed terms

10

Results Retrieved

From Wikipedia

B

Bharat Smart Search

DSA-Powered Search Engine

[Home](#)[Performance](#)[DSA](#)[How It Works](#)

CORE DATA STRUCTURES & ALGORITHMS

DSA Algorithms Used

Every stage of the search pipeline uses a specific data structure or algorithm.

01

HashMap

DOCUMENT STORAGE

Stores document IDs and their corresponding document objects for fast lookup.

Lookup: $O(1)$ average

02

Inverted Index

KEYWORD RETRIEVAL

Maps each keyword to the documents where that keyword occurs and stores its frequency.

Frequency: Search

Fast keyword lookup

03

Merge Sort

RESULT RANKING

Sorts matching documents by their keyword frequency to produce ranked results.

Time: $O(n \log n)$

04

Trie

PREFIX SUGGESTIONS

Stores words character-by-character and supports prefix-based search suggestions.

Prefix Search: $O(1)$

SEARCH ALGORITHM PIPELINE

01 Text Processing → 02 HashMap → 03 Inverted Index → 04 Frequency Match → 05 Merge Sort → 06 Ranked Results

B

Bharat Smart Search

DSA-Powered Search Engine

[Home](#)[Performance](#)[DSA](#)[How It Works](#)

SEARCH RESULTS

Knowledge Found

10 results • Completed in 1076.95 ms

01

India (disambiguation)

Frequency: 6 Inverted Index Merge Sort

Look up India, India, India, India, or India in Wiktionary, the free dictionary. India is a country in South Asia. India may also refer to: India, a village

HashMap Frequency Ranking

OPEN WIKIPEDIA ...

02

India

Frequency: 3 Inverted Index Merge Sort

India, officially the Republic of India, is a country in South Asia. It is the world's seventh-largest country by area and the largest by population.

HashMap Frequency Ranking

OPEN WIKIPEDIA ...

03

History of India

Frequency: 3 Inverted Index Merge Sort

region of India, modern day Punjab from Central Asia in several waves of migration. The Vedic Period of the Vedic people in northern India (1500-500 BCE)

HashMap Frequency Ranking

OPEN WIKIPEDIA ...

04

Indian National Developmental Inclusive Alliance

Frequency: 2 Inverted Index Merge Sort

Political party, Bharat (BJP) is a Hindu nationalist political alliance of several political parties led by the Bharat (BJP) party

HashMap Frequency Ranking

OPEN WIKIPEDIA ...

B

Bharat Smart Search

DSA-Powered Search Engine

[Home](#)[Performance](#)[DSA](#)[How It Works](#)

05

Demographics of India

[Frequency: 2](#)[Inverted Index](#)[Merge Sort](#)

India is the most populous country in the world, comprising approximately one-sixth of the world population. According to the United Nations' World Population

[HashMap](#)[Frequency Ranking](#)

OPEN WIKIPEDIA ...

06

East India Company

[Frequency: 2](#)[Inverted Index](#)[Merge Sort](#)

The East India Company (EIC) was an English, and later British, joint-stock company that was founded in 1600 and dissolved in 1874. It was formed for

[HashMap](#)[Frequency Ranking](#)

OPEN WIKIPEDIA ...

07

Economy of India

[Frequency: 2](#)[Inverted Index](#)[Merge Sort](#)

India has a developing market-oriented economy with substantial state participation in strategic sectors. It is the world's sixth-largest economy by nominal

[HashMap](#)[Frequency Ranking](#)

OPEN WIKIPEDIA ...

08

Languages of India

[Frequency: 2](#)[Inverted Index](#)[Merge Sort](#)

Languages of India belong to several language families, the major ones being the Indo-Aryan languages spoken by 78.05% of Indians and the Dravidian languages

[HashMap](#)[Frequency Ranking](#)

OPEN WIKIPEDIA ...

B

Bharat Smart Search

DSA-Powered Search Engine

[Home](#)[Performance](#)[DSA](#)[How It Works](#)

09

List of highest-grossing Indian films

[Frequency: 1](#)[Inverted Index](#)[Merge Sort](#)

century. Since 2003, there are markets in over 90 countries where films from India are screened. During the first decade of the 21st century, there was a steady

[HashMap](#)[Frequency Ranking](#)

OPEN WIKIPEDIA ...

10

India

[Frequency: 0](#)[Inverted Index](#)[Merge Sort](#)

India is the fourth studio album by Brazilian singer Gai Costa, released on 1973 by Philips Records. Its major hits were "India", "Volta" and "Desafinado"

[HashMap](#)[Frequency Ranking](#)

OPEN WIKIPEDIA ...

SYSTEM ARCHITECTURE

How Bharat Smart Search Works

A combination of real Wikipedia data and Java data structures creates the search engine.

01

User Query

User enters a keyword or multiple keywords.

02

Wikipedia

Real documents are retrieved through the API.

03

Java DSA

HashMap and Inverted Index find matches.

04

Ranking

Merge Sort ranks results by frequency.

🌐 16 languages ▾

Read

Read

Read

Read

From Wikipedia, the free encyclopedia

This article is about the highest-grossing films made in India. For the highest-grossing films in the box office territory of India, see [List of highest-grossing films in India](#).

This ranking lists the **highest-grossing Indian films** produced by **Indian cinema**, based on conservative global **box office** estimates as reported by organisations classified as green by Wikipedia.^[a] The figures are not adjusted for inflation. However, there is no official tracking of figures and sources publishing data are frequently pressured to increase their estimates.^[1]

Indian films have been screened in [markets around the world](#) since the early 20th century.^[2] Since 2003, there are markets in over 90 countries where films from [India](#) are screened.^[3] During the first decade of the 21st century, there was a steady rise in the ticket price, a tripling in the number of theatres and an increase in the number of prints of a film being released, which led to a large increase in the box office collections.^[4]

As of 2014, *Hindi cinema* represents 43% of the net box office revenue in India, while *Telugu* and *Tamil cinema* represent 36%, and other industries constitute 21%.^[5] As of 2020, the combined revenue of all other language film industries has surpassed that of the Hindi film industry.^[6] By 2021, Telugu cinema became the largest film industry of India in terms of box-office.^{[7][8][9]} In 2023, the Hindi film industry accounted for 44% of box office revenue, followed by the Telugu industry at 19% and the Tamil industry at 16%. The *Kannada* (5%), *Malayalam* (3%), *Bengali*, *Marathi*, *Odia*, *Punjabi*, *Gujarati* and *Bhojpuri* industries contributed to the remainder, while the foreign film industry made up 9% of the total revenue.^[10] In 2024, the Hindi and the Telugu film industries accounted for 40% and 20% of the domestic gross box office revenue respectively, followed by Tamil with 15% share and Malayalam with 10% share.^[11]

Appearance

Text

☐ Small

 Standard☐ Large

Width

 Standard

☐ Wide

Color

☐ Automatic

 Light

☐ Dark

- Contents** [hide](#)

(Top)

 - Overview
 - Key
 - Highest-grossing films
 - ▼ Highest-grossing films by language
 - Assamese
 - Bengali
 - Bhojpuri
 - Gujarati
 - Hindi
 - Kannada
 - Malayalam
 - Marathi
 - Meitei
 - Odia
 - Punjabi
 - Tamil
 - Telugu
 - Highest-grossing films